A Machine-Checked No-Go Certification for the Six-Qubit Moment Orbits

Formalization notes

Abstract

Fifty-one modules of this development each carry a Lean 4 proof, checked by the Lean kernel alone: their axiom footprint is exactly `[propext, Classical.choice, Quot.sound]`, with no appeal to `native_decide` and no `sorry` anywhere in the development. This note describes what was certified, the standard the certification was held to, the architecture of the development, the handful of algebraic mechanisms that between them close all fifty-one cases, and how to reproduce every step. It also reports, in Section 6, the reasoning errors that were made and caught during the work, because the way they were caught is the most transferable part of the exercise.

Scope, stated precisely. Each orbit carries two theorems. The first, `no_go_of_moments`, takes its Knill–Laflamme rows as hypotheses and derives a contradiction. The second, `no_code`, assumes only that the two logical states are supported on the residue-0 and residue-δ fibers, are normalized, and satisfy `SS.HasDistance ![c, d] 3`; every Knill–Laflamme row is then *derived* through `SixQubit/CodeBridge.lean`. So the development does establish, inside the kernel, that no exact $((6, 2, 3))$ code is supported on any of these fifty-one orbits. What it does *not* yet establish is that every code must be supported on one of them: the reduction of an arbitrary code to one of the sixty-six rank-compatible moment orbits, and the transport of other angle records to the chosen representatives, are not formalized. Section 9 states this boundary.

Contents

1 What is certified **1**
- 1.1 The statement . . . 1
- 1.2 The certification standard . . . 2
- 1.3 The result . . . 2
- 1.4 Toolchain . . . 3

2 Architecture **3**
- 2.1 The `SS` layer . . . 3
- 2.2 Transcription modules . . . 3
- 2.3 Algebra modules . . . 3
- 2.4 The audit module . . . 4

3 The mechanisms **4**

4 The four hard orbits **5**
- 4.1 Orbit 33: a block reflection . . . 5
- 4.2 Orbit 29: a K_4 of orthogonalities . . . 5
- 4.3 Orbit 31: a 4×4 table . . . 5
- 4.4 Orbit 26: the cross-anchor contraction . . . 6

5 A tooling bug that gated three orbits **6**

6 Errors made, and how they were caught **7**

7 Reproduction **8**
7.1 Checking the proofs 8
7.2 Regenerating the Lean 8
7.3 Re-running the independent checks 9
7.4 The workflow records 9

8 Repository layout **9**

9 Open items **9**

1 What is certified

1.1 The statement

Fix q and an angle vector $a \in (\mathbb{Z}/q)^6$. The residue fibers are

$$S_0 = \{x \in \mathbb{F}_2^6 : \textstyle\sum_i a_i x_i \equiv 0 \bmod q\}, \qquad S_1 = \{x \in \mathbb{F}_2^6 : \textstyle\sum_i a_i x_i \equiv \delta \bmod q\},$$

and a putative code supported on this orbit is a pair of unit vectors

$$|0_L\rangle = \sum_{x \in S_0} c_x |x\rangle, \qquad |1_L\rangle = \sum_{y \in S_1} d_y |y\rangle.$$

Distance three means the Knill–Laflamme conditions hold for every Pauli P of weight at most two:

$$\langle 0_L|P|1_L\rangle = 0, \qquad \langle 0_L|P|0_L\rangle = \langle 1_L|P|1_L\rangle.$$

For each of the fifty-one orbits the development proves that this system has no solution. The Lean statement is a theorem named `no_go_of_moments` in each orbit's module, of the shape

```
theorem no_go_of_moments (c d : SS.BitString 6 -> C)
    (hm : MomentSystem (normSq (c y0)) ... (normSq (d z15)))
    (E_ : offSum c d ef_ w_ = 0) ...
    (D_ : diagBot c ef_ w_ = diagTop d ef_ w_) ...
    : False
```

where `offSum` is the off-diagonal Knill–Laflamme sum for a flip mask and a Z-dressing, `diagBot`/`diagTop` are the two sides of a diagonal condition, and `MomentSystem` is the linear system of pure-Z diagonal conditions together with the two normalizations and the nonnegativity of every probability.

1.2 The certification standard

Five rules were fixed at the outset and held for all fifty-one orbits.

Rule 1. The transcription of the two fibers is certified by `decide` over all sixty-four bit strings, not asserted. Each module proves `S0_eq_fiber` and `S1_eq_fiber`, which state that the listed support is exactly the residue fiber.

Rule 2. Moment data are derived, not assumed. Every probability identity used in a proof is a named theorem taking `MomentSystem` as its only hypothesis and closed by a single `linear_combination` whose coefficients come from an exact rational reduction against the moment matrix.

Rule 3. The equations are generated by script from verified matrices. Incidence lists, Walsh sign patterns, and certificate coefficients are computed in Python from the fiber data and emitted as Lean; they are never typed by hand.

Rule 4. The axiom footprint of every headline theorem is exactly `[propext, Classical.choice, Quot.sound]`. In particular `Lean.ofReduceBool`, the signature of `native_decide`, never appears.

Rule 5. The build is clean: zero errors and zero warnings, with no linter suppressed.

Rule 4 is the load-bearing one. `#print axioms` is transitive, so a single clean line for `no_go_of_moments` certifies the entire dependency cone beneath it.

1.3 The result

Orbits certified	51 (every no-go orbit)
Headline `no_go_of_moments` theorems	51
Lines in the orbit modules	41 521
Theorems in the orbit modules	12 521
Lines in the whole development	90 109
Audit entries in `SixQubit/Audit.lean`	332
Audit entries with a non-clean footprint	0
Build errors	0
Build warnings	0
Occurrences of `sorry` in tactic position	0
Occurrences of `native_decide` in tactic position	0

The certified orbits are exactly the fifty-one the manuscript excludes: 5, 6 and 47–52 at projective order three; 12–15, 17–20, 24, 41–44 and 46 at order two; 25–32 and 62–63 at order four; 33–35 and 37–40 at order five; and 53–61 and 64–66 at orders six through nine. Orbit 66 lives in `SixQubit/Order9.lean` for historical reasons and all others in `SixQubit/Orbit<n>.lean`. The remaining fifteen of the sixty-six rank-compatible orbits all target the realized orders $\{2, 3, 5\}$ and are not excluded; four of them (7, 8, 9 and 36) are known to be realized. A per-orbit table, generated from the sources, is in `results/orbit-table.tsv`.

1.4 Toolchain

Lean	`leanprover/lean4:v4.27.0`
Mathlib	`fc278a179e431ed10630af63fc05481680c0c7bd`

The pins are recorded in `lean-toolchain` and `lake-manifest.json`. Nothing outside Mathlib is used mathematically; the other manifest entries are Mathlib's own dependencies.

2 Architecture

The development is layered, and the layering is what made the harder orbits tractable.

2.1 The `SS` layer

`SS/` (1 608 lines) holds the problem-independent infrastructure: bit strings, Pauli strings, the enumeration of admissible weight-≤ 2 Paulis, and the predicates that decide which Knill–Laflamme conditions are automatic for a given support. Nothing here is orbit-specific.

2.2 Transcription modules

Each orbit has a module containing:

- the two fibers as explicit `def`s with a `decide` proof that they are the residue fibers;
- the `MomentSystem` structure for that orbit, generated from the exact rational moment matrix;
- the derived moment facts, each a theorem over `MomentSystem`;
- `offSum`, `diagBot`, `diagTop`, the incidence/flip/sign lemmas (all by `decide`), and the Walsh expansion lemmas;
- the orbit's Knill–Laflamme relations, each proved by `linear_combination` from expansions with script-computed coefficients;
- the headline `no_go_of_moments`.

2.3 Algebra modules

Thirteen algebra modules carry the mathematics, separated from the bit strings entirely. The first four served the hard orbits of the initial batch:

Module	Serves	Lines
`SixQubit/Block33.lean`	orbit 33	327
`SixQubit/Pair29.lean`	orbit 29	636
`SixQubit/Table31.lean`	orbit 31	490
`SixQubit/Block26.lean`	orbit 26	415

Nine more were added later: `Triangle` and `Frame` (shared by several orbits), `Spinor`, `ParityFour`, `Hafnian`, `Trace5`, `Flip5`, `Kernel5`, and `K5` for the rebuilt orbit 30.

These modules mention no bit strings at all. They take complex numbers and the relations among them and return `False` or an algebraic conclusion. This has a practical consequence worth stating plainly: *the mathematical core of a hard orbit can be written and compiled before any of the transcription plumbing exists*, which is how the risky part gets de-risked first. It also means the algebra can be audited on its own, without wading through several hundred `by decide` lemmas.

2.4 The audit module

`SixQubit/Audit.lean` imports every orbit module and runs `#print axioms` on each headline theorem and on selected intermediate lemmas. Its output, captured in `results/axiom-audit.txt`, is 332 lines, every one reading

```
'SixQubit.<Module>.<theorem>' depends on axioms:
  [propext, Classical.choice, Quot.sound]
```

3 The mechanisms

Thirty-five of the fifty-one orbits fall to one of a small ladder of mechanisms, each of which is driven by a script that finds the certificate and emits the Lean. The ladder, in increasing order of strength:

1. **k-edge Walsh.** A flip mask whose incidence set is small enough that the dressing sign matrix forces one incidence product to vanish, while the moment polytope forces both endpoints to have positive probability.
2. **Support collapse.** A coordinate forced to zero by the polytope removes incidences, which exposes new forced zeros; iterate.
3. **Propagation.** Two-term relations chain across masks.
4. **Quadratic closure.** The squared-modulus consequences of the two-term relations, reduced against the moment system, sometimes linearize and pin the probabilities outright.
5. **Positive row sum.** A shared lemma turning a row of the moment matrix into a contradiction with nonnegativity.
6. **Phase-ratio transitivity.** The quantities $\rho_x = \overline{c_x} d_{\hat{x}}$ are linked across masks; connectivity forces a common phase.
7. **Star hub and mirror permanent.** A hub vertex plus the vanishing of a 2×2 permanent (not determinant — see below).
8. **Case analysis on the active set.** Driven by the polytope's face structure.
9. **Hub without a star.** A vanishing-pair tree replaces the hub.
10. **Diagonal Knill–Laflamme conditions.** The last rung; orbit 25 was the first orbit that could not be closed without them.

A recurring trap deserves its own paragraph, because it caught this work three times. When the fibers admit an involution, a four-term Walsh sum collapses to

$$(1 + \sigma_x\sigma_y)\, P(x, \hat{y}) + (1 + \sigma_z\sigma_w)\, P(z, \hat{w}),$$

which is a genuine cross relation only when *both* sign products are $+1$ and is vacuous otherwise. With the two pairs' relative sign flipped one obtains the 2×2 *determinant* rather than the *permanent*, and the determinant version is satisfied by any rank-one grid, so it yields no contradiction at all. Any argument that reads a permanent where the data supply a determinant is worthless, and the failure is silent.

4 The four hard orbits

Four orbits needed genuinely new arguments. In each case the argument obtained here is substantially shorter than the one in the manuscript, and in two cases it avoids machinery the manuscript needs.

4.1 Orbit 33: a block reflection

$a = (0, 1, 1, 3, 3, 3)$, $q = 5$, fibers $12 + 12$. The first qubit carries angle zero, so both fibers are closed under flipping it and split into six blocks of two; write $c_i, d_i \in \mathbb{C}^2$.

The moment system alone pins every block norm to $R = (3, 2, 2, 1, 1, 1)/10$. The diagonal conditions for F on the zero-angle qubit together with FZ_j on a second qubit give $Cf = JCg$, where C is the 6×6 tail-sign matrix ($\det C = 160$) and $J = \mathrm{diag}(1, -1, \ldots, -1)$; and

$$C^{-1}JC = 2R\mathbf{1}^T - I,$$

which is equivalent to the six matrix identities

$$c_i c_i^\dagger + d_i d_i^\dagger = 2R_i\rho.$$

The cross conditions make blocks $1, 2$ collinear with block 0 on both fibers, which forces $c_j^\dagger d_0 = 0$ and $c_0^\dagger d_j = 0$ for $j = 3, 4, 5$. Orthogonality in $\mathbb{C}^2$ then gives $\rho = I/2$ and simultaneously $\rho = \frac{10}{3} c_0 c_0^\dagger$, so $c_0 c_0^\dagger = \frac{3}{20} I$, whose off-diagonal entry is zero while $\mathrm{normSq}(p_0)\,\mathrm{normSq}(q_0) = 9/400$.

Everything is division free; the only case analysis is `mul_eq_zero` against explicit nonzero rationals.

4.2 Orbit 29: a K_4 of orthogonalities

$a = (1, 1, 1, 1, 2, 2)$, $q = 4$, fibers $16 + 16$, and the two fibers are *not* complementary — a fact that breaks the tooling used elsewhere and required a separate relation extractor (`tools/core/relgen.py`).

Both fibers are closed under $X_5 X_6$, so each splits into eight pairs. Read a pair as a vector in $\mathbb{C}^2$; then every two-term relation is an orthogonality $C_m^\dagger D_n = 0$, and the twelve that survive realise exactly the six edges of K_4 on the four top pairs that carry them.

Two observations shortened the proof considerably relative to the manuscript's fifteen-face audit.

- The uniform identity $P_{\{n,n'\}} = Q_n + Q_{n'} - \frac{1}{4}$ collapses the face table into a single lemma with one hop.
- One never picks a nonzero vector. Instead the common bottom direction is *constructed* from the anchor: $u := (-\overline{v_1}, \overline{v_0})$. It is automatically nonzero, automatically orthogonal to v, and $u \perp D_n$ is literally $\det[D_n, v] = 0$. Only two case splits survive.

4.3 Orbit 31: a 4×4 table

$a = (1, 1, 1, 2, 2, 2)$, $q = 4$, fibers $16 + 16$ and complementary. The last three angles are all 2, so both fibers are stabilised by the Klein four-group on those qubits and split into four cosets of four. The sixteen positions form a 4×4 table with rows A, B_1, B_2, B_3 and columns $e_0, \ldots, e_3$.

The moment normal form — all of it verified exactly against the moment system — is

$$p_1(\bar{v}) = p_0(v) \text{ at every position}, \quad \sum_e \beta_{i,e} = \tfrac{1}{4}, \quad \alpha_e = \sum_i \beta_{i,e} - \tfrac{1}{8}.$$

The first of these lets every active position be written $d_{\bar{v}} = \rho_v \overline{c_v}$ with $|\rho_v| = 1$, and twelve two-term relations make ρ constant along *columns*.

Two facts remove the phase from the Lean side entirely, which is what made this orbit finishable:

- $v_e = \overline{d_{\hat{A}_e}}$: the kernel vector is literally the conjugate of the top amplitude at the complement of A_e. No division, no phase object.
- Two columns share a phase exactly when $v_e a_f = v_f a_e$, a division-free complex equation one can `by_cases` on.

Family one then says the four bottom columns are dependent with coefficients v, so pairing against each column gives *four* equations $\sum_e v_e M_{ef} = 0$, where M_{ef} is the Gram matrix of the B columns. Family three pins $M_{ef} = \overline{v_e} v_f$ whenever the phases differ; the division-free form of that statement, and the fact that closed the last gap, is the unconditional identity

$$\big(M_{ef} - \overline{v_e} v_f\big)\big(v_e \overline{v_f} - a_e \overline{a_f}\big) = 0.$$

Its second factor packages $a_e \neq 0$, $a_f \neq 0$ and $\rho_e \neq \rho_f$ into one side condition. A column alone in its phase class then has bracket $3/8$ and dies at once, and the five branches of the phase partition close in turn.

For comparison, the manuscript closes this orbit by a computer-assisted rational classification of sixteen support types.

4.4 Orbit 26: the cross-anchor contraction

$a = (0, 1, 1, 2, 2, 3)$, $q = 4$, fibers $16 + 16$. Blocks pair into four groups of two; the group $\{6, 7\}$ is the anchor and carries probability $1/8 + 1/8$.

Fix an anchor block j, set $X_i = C_j^\dagger D_i$, and let $R_g = (X_{2g}, X_{2g+1})$. Contracting the outer identity with that anchor gives $R_g J R_h^T = 0$ for the three pairs $g < h$, with J the *symmetric* antidiagonal form; three nonzero pairwise J-orthogonal rows in $\mathbb{C}^{1\times 2}$ lie on one coordinate axis.

The axis, however, is chosen *per anchor*, and an earlier version of this argument silently used both anchors at once. That is a real gap: an adversarial review produced explicit witnesses in which $C_6 \not\parallel C_7$ and the counting closes with slack. The repair is to contract the outer identity with *different* anchors in the two slots, on the a index with D_7 and on the b index with $\overline{C_6}$, which yields

$$R_g^{(7)} J R_h^{(6)T} + R_h^{(7)} J R_g^{(6)T} = 0,$$

reading $a_g b_h + a_h b_g = 0$ in the mixed configuration — whose only solution has one of the six entries zero. Two caveats, both checked rather than assumed: the cross relation is *vacuous* in the aligned branch, which is closed separately by a moment and Cauchy–Schwarz argument; and the fact

$$P_0 + P_2 + P_4 = P_1 + P_3 + P_5 = \tfrac{3}{8}$$

is load-bearing — an ablation shows that dropping it makes the whole fact set feasible again, precisely at the aligned point $P = (0, \frac{1}{4}, 0, \frac{1}{4}, 0, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$.

5 A tooling bug that gated three orbits

The scan tooling under-generated the admissible Pauli set. For a weight-one flip e it returned only $\{0\} \cup \{e_j\}$, seven strings, where the correct set is $\{0, e_i\} \times \{0, e_j\}$, twelve; it thereby dropped every $Y_i Z_j$. For $e = 0$ it returned nothing at all, dropping all twenty-two pure-Z conditions. The correct total over all $|e| \leq 2$ is

$$154 = 1 + 6 \cdot 3 + 15 \cdot 9,$$

the number of weight-≤ 2 Pauli operators on six qubits, which is the check that should have been run at the start.

Three of the last four orbits were blocked by this. Orbit 33's argument needs $F \otimes Z_j$ for F ranging over I, X, Y, Z on the zero-angle qubit, and three of those four families were missing.

No earlier certification was invalidated. Those proofs used a *subset* of the true conditions, which makes them harder, never wrong. The fix lives in `tools/core/kl.py`, which enumerates all sixty-four dressings and filters on $|\operatorname{supp}(e) \cup \operatorname{supp}(z)| \leq 2$ rather than being clever.

6 Errors made, and how they were caught

Five reasoning errors were made during this work and all five were caught before anything unsound entered the development. They share one shape: *asserting that a relation has a certain form without checking that form against the originals.* Recording them, and the detection methods, is more useful than pretending the path was straight.

1. **A wrong δ.** Choosing the second fiber by size alone made the propagation report four orbits "closed" when their polytopes were simply infeasible under a wrong δ. Fixed by matching the polytope dimension d_{LP} as well — but *only for the orbits then outstanding.* Orbits 30, 53, 55 and 65 had already been certified and were never re-checked, and all four turned out to use a residue of additive order two instead of the order their projective order demands. A collaborator's audit found this; all four were rebuilt at the correct residue, and the correct fibers are disjoint from the ones the old proofs used. The guard that should have existed from the start is now `tools/manifest.json` plus `tools/verification/gate.py`, which derives $\delta = q/m$ from the paper's own table and refuses any module whose residue, angle vector, fiber or moment rows disagree with it.

2. **The collapse trap, three times.** A four-term sum was read as a cross relation when the sign products made it vacuous, or as a permanent when the data gave a determinant. Fixed by writing an extractor (`tools/core/relations.py`) that computes every two-term relation genuinely in a mask's row space.

3. **Orbit 26: per-edge matrix relations that do not exist.** The weight-one masks carry eight incidences with rank only four and admit *no* two-term relation; only group-level sums exist.

4. **Orbit 31: a family-three statement pushed too strong.** Claiming each term vanishes separately gives $G_{ef}t_{ef} = 0$, which is false. The bottom sign is $(-1)^{z\cdot x}$ and the A row sits at complementary last-three-bits from the B rows, so an odd-weight dressing flips the A-row sign relative to the others.

5. **Orbit 31: a covering count with a counterexample.** The claim that covering six column pairs with three rows forces one nonzero entry per row is false: $Z = \{0,1,2\}, \{0,1,3\}, \{2,3\}$ covers all six with empty intersection. The branch does close, but by the full permanent condition, not by counting.

The detection methods, in increasing order of usefulness:

1. **Row-space membership.** Reduced row echelon form against the mask's dressing sign matrix answers "is this relation actually available?" directly. This caught error 3.

2. **Replay against the originals.** Take a solution of the reduced system and evaluate the *raw* Knill–Laflamme conditions on it. Exactly two failed for error 4, and both were weight-one dressings, which pointed straight at the sign. This is cheap and should be run on every reduction.

3. **Adversarial review.** A multi-agent pass whose agents were instructed to *refute* rather than confirm produced explicit witnesses for error 3's consequence in orbit 26, including a configuration satisfying every fact the argument used.

4. **Writing the Lean.** Error 5 surfaced while formalising the branch: the combinatorial step had to be made concrete, and the counterexample fell out. This is the cheapest of the four and argues for a working practice: *do not derive at length on paper and formalise in one batch; formalise as you go.*

For the final two orbits the work was delegated to multi-agent workflows under three non-negotiable instructions: verify by machine rather than assert, actually compile rather than report compiling, and classify every hypothesis of the headline theorem. That last one is the important one. A headline `False` theorem is only as good as its hypotheses, and a proof becomes a shell the moment an algebraic fact that should have been derived is smuggled in as an assumption — a defect invisible from the outside. Both final headlines were classified by hand after the fact:

Orbit	Hypotheses	offSum = 0	diagBot = diagTop	MomentSystem	Other
31	46	33	12	1	0
26	41	24	16	1	0

7 Reproduction

7.1 Checking the proofs

From the repository root:

```
lake build
```

should report zero errors and zero warnings. Then

```
lake build SixQubit.Audit
```

prints the 332 axiom lines; every one must read `[propext, Classical.choice, Quot.sound]`. A stored copy of the expected output is `results/axiom-audit.txt`, so

```
lake build SixQubit.Audit 2>&1 | grep "depends on axioms" \
  | sed 's/^info: [^ ]* //' | diff - results/axiom-audit.txt
```

should produce no output.

To confirm the absence of the forbidden constructs:

```
grep -rn "sorry\|native_decide" SixQubit/ SS/
```

Every hit must be inside a docstring; there are thirty-three of them, of which twenty-eight are the sentence stating that `native_decide` is not used.

7.2 Regenerating the Lean

The transcription modules are machine-generated. The generators live in `tools/generators/`; each reads the orbit data from `tools/orbits/` and the shared infrastructure from `tools/core/`, and writes a `.lean` file. They require Python with `sympy`, `numpy` and `scipy`. Set `PYTHONPATH` to include `tools/core` and `tools/orbits`, or run from the repository root with

```
python -c "import sys; sys.path[:0] = ['tools/core','tools/orbits']; ..."
```

7.3 Re-running the independent checks

`tools/verification/` holds the scripts that answer questions the Lean proofs do not: whether an orbit is excluded numerically at all, which relations are genuinely available, and which branches of a case analysis are live. Highlights:

Script	What it answers
`feas2.py`	Is the full weight-≤ 2 system numerically unsatisfiable?
`close31.py`	Orbit 31 against the raw conditions, phases free
`branch31.py`	Which phase partitions of orbit 31 are excluded
`collapse29.py`	Orbit 29's support collapse
`stepA\_*.py`	The four silent failure modes behind orbit 26's Step A
`t26\_*.py`, `x26\_*.py`	Orbit 26 facts, ablations, branch tree

`feas2.py` includes certified orbits as controls: an orbit with a kernel-checked no-go proof must come out excluded, and it does (orbit 25 gives a best residual of 0.79). Running it without the controls would not be a test.

7.4 The workflow records

`results/workflows/` holds the scripts of the multi-agent passes and their verdicts, including the adversarial review that found the orbit 26 gap and the audits of the final two assemblies. They are kept because they contain the negative results — which relations are *not* available, which branches are vacuous — that are expensive to rediscover and easy to assume away.

8 Repository layout

Path	Contents
`SixQubit/`	the fifty-one orbit modules, thirteen algebra modules, the audit
`SS/`	problem-independent bit-string and Pauli infrastructure
`formalization/`	this document and the reader's guide
`tools/core/`	fibers, Pauli enumeration, moment matrices, relation extractors
`tools/orbits/`	per-orbit data and block structure
`tools/generators/`	the scripts that emit the Lean modules
`tools/verification/`	numerical and symbolic checks independent of Lean
`results/`	axiom audit, orbit table, workflow records

9 Open items

One item concerns the formalization itself.

1. **The classification is not formalized.** Neither the reduction of an arbitrary code to one of the sixty-six rank-compatible moment orbits, nor the equivalence of other angle records to the chosen representatives, has a Lean proof.

Two further items concern the manuscript rather than the formalization.

1. Whether the equations and probability data developed here should be folded back into the manuscript. The arguments obtained for orbits 26, 29, 31 and 33 are shorter than the manuscript's and in two cases avoid machinery it uses (matrix inverses and a genericity argument for orbit 26; a sixteen-type support classification for orbit 31), so there is a case for replacing rather than supplementing.

2. In a companion manuscript, `A_dyadic_ladder/paper/main.tex`: the sentence at line 1045 asserting that no lower bound is known is not correct for stabilizer codes, and the table caption at line 1000 needs revision. Neither touches the present development.